The Journal of Open Source Software

# *ai3:* A Framework Enabling Algorithmic Selection in Deep Neural Networks

**Timothy L. Cronin IV**[1] **and Sanmukh R. Kuppannagari**[1]

**1** Department of Computer and Data Sciences, Case Western Reserve University

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:**

**Submitted:** 20 October 2024
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

## Summary

*ai3*, initially proposed in (Cronin IV & Kuppannagari, 2024), is an open source framework providing fine grain algorithmic control over an existing deep neural network (*DNN*) in both its training and inference stage. *ai3* provides high performance, accelerated *C++* implementations of various algorithms to complete common deep learning operations. Additionally, *ai3*, enables users to implement their own *C++* implementations which are included in the package upon installation. Both the user defined and builtin implementations can be selected to complete the operations making up a *DNN*. Selection and altering of the *DNN* is done via an easy to use *Python* interface 1. *ai3*, is an abbreviation of algorithmic innovations for accelerated implementations of artificial intelligence, reflecting the accelerated implementations provided by and the potential for algorithmic innovations when using the framework.

## Statement of Need

*DNNs* have demonstrated significant success across many domains. This success has been accompanied by substantial improvements to the algorithms used to perform the operations forming the *DNN*. However, the most efficient algorithm for any given operation of a *DNN* depends on a high number of factors such as the use case of the *DNN*, input size, operation hyper-parameters and hardware available. Due to this, discovering the best performing algorithm is a difficult and time consuming task. Additionally, altering the algorithms in use by a *DNN* is a difficult task requiring requiring algorithm development and hardware mapping expertise. This makes such exploration and its benefit of improved performance inaccessible to a significant portion of users who come from scientific and engineering domains but may not be experts in computer science.

## Framework Overview

The framework provides two functions to perform algorithmic selection and swapping. One function creates a new equivalent *DNN* with all operations from the original *DNN* converted to the frameworks implementations of the selected algorithms. The second function swaps, in place, a specific operation out of the existing *DNN* for the frameworks implementation of the selected algorithm. The function which swaps a specific operation attempts to make the new operation integrate well with the *DNNs* original framework. For example, after swapping the operations within a *DNN* built with *PyTorch* (Ansel et al., 2024), the *DNN* can still be trained and compiled for execution in *PyTorch's* graph mode. Both of these functions are illustrated in figure 1.

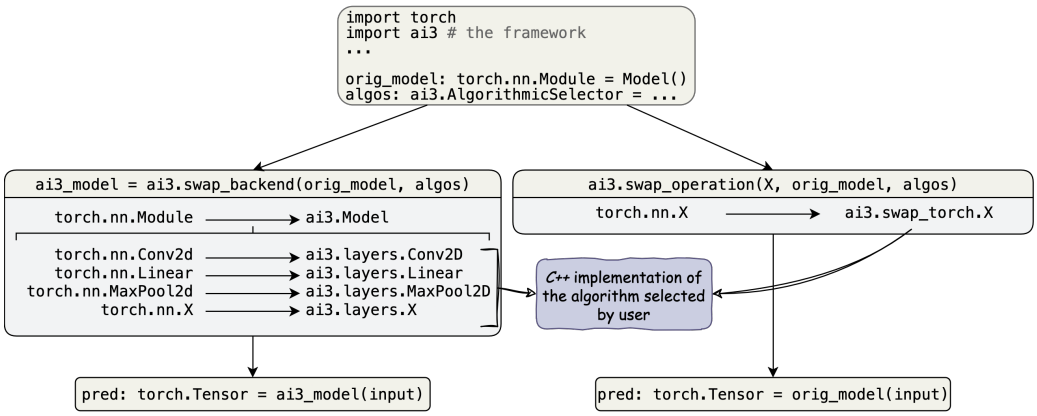


**Figure 1:** Possible Paths of a *DNN* Through *ai3*

Both of these functions have an optional parameter for the algorithmic selector, if none is passed then a default algorithm is selected by the framework. The function converting every operation receives a mapping from operation type to the selector and the function swapping one operation receives a single algorithmic selector for the operation being swapped. There are multiple types of algorithmic selectors. Possible options include a single algorithm which is used for all instances of the operation or a list of algorithms where each operation uses the algorithm with the same index as that operation has relative to other operations of the same type. Another option is a function which returns the algorithm to use. The function is called by the framework with the original operation as the parameter, the function will also be passed the input shape if an input shape is provided to the function providing the algorithmic selection.

# Code Example

This example demonstrates use of both of functions on a *VGG-16* model provided by the *torchvision* (maintainers & contributors, 2016) package. In the call to `convert`, a mapping is passed from operation to algorithmic selector. When performing the swaps for convolution, the operation and the shape of the input to that operation are passed to a function which analyzes those parameters and returns the algorithm to use. For all other operations the algorithm used is the default provided by the framework. In the `swap_operation` call, the `torch.nn.Conv2d` type is passed to swap out convolutional layers, a list of 16 elements alternating between `direct` and `SMM` (Ofir & Ben-Artzi, 2022) is passed meaning the 16 convolution layers alternate between the `direct` and `SMM` algorithms to perform the convolutions.

```
import torch
import torchvision
import ai3

def conv2d_selector(orig, input_shape) -> str:
    out_channels = orig.weight.shape[0]
    if (out_channels < 50 and
        input_shape[1] < 50 and
        input_shape[1] > 150 and
            input_shape[2] > 150):
        return 'direct'
    return 'smm'

input_shape = (1, 3, 224, 224)
input_data = torch.randn(input_shape)
```

DRAFT

```
vgg16 = torchvision.models.vgg16(
    weights=torchvision.models.VGG16_Weights.DEFAULT)
vgg16.eval()
with torch.inference_mode():
    orig_out = vgg16(input_data)
    model: ai3.Model = ai3.convert(
        vgg16, {'conv2d': conv2d_selector,
                'maxpool2d': 'default'},
        input_shape)
    sb_out = model(input_data)
    ai3.swap_operation(torch.nn.Conv2d, vgg16, ['direct', 'smm'] * 8, input_shape)
    sc_out = vgg16(input_data)
    assert torch.allclose(
        orig_out, sb_out, atol=1e-4)
    assert torch.allclose(
        orig_out, sc_out, atol=1e-4)
```

**Code Sample 1:** Use of Both Functions


# Acknowledgements

This work is sponsored by the *U.S.* National Science Foundation under award numbers 2117439, 2411447, and 2425535.


# References


Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., … Chintala, S. (2024, April). PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. https://doi.org/10.1145/3620665.3640366

Cronin IV, T. L., & Kuppannagari, S. (2024). *A framework to enable algorithmic design choice exploration in DNNs*. https://arxiv.org/abs/2410.08300

maintainers, T., & contributors. (2016). TorchVision: PyTorch's computer vision library. In *GitHub repository*. https://github.com/pytorch/vision; GitHub.

Ofir, A., & Ben-Artzi, G. (2022). SMM-conv: Scalar matrix multiplication with zero packing for accelerated convolution. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3066–3074. https://doi.org/10.1109/CVPRW56347.2022.00346


DRAFT